

P O R T F O L I O

박인호 | Game Programmer

NPC 미션 시스템 + 퀘터뷰 디펜스 미니게임 예제

Unity 6 (6000.3.10f1) Environment | **Factory Method** · **State Machine** · **Observer** · **Object Pool** · **Template Method** Applied

MissionSystem_Unity

MISSION SYSTEM DESIGN GOALS

</>

UI와 도메인 로직의 완전한 분리

핵심 연산과 연출의 독립으로 유지보수 비용 절감

</>

객체 간 직접 참조 최소화

의존성을 제거하여 예기치 못한 사이드 이펙트 방지

</>

상태 기반(State-driven) 설계

미션 라이프사이클의 명확한 관리와 흐름 제어

</>

런타임 비용 최소화

오브젝트 풀링으로 Instantiate/Destroy 비용과 GC 부담 절감

</>

확장성 및 유지보수성 확보

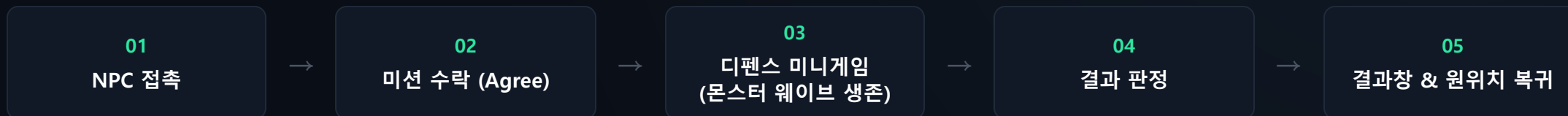
새로운 미션 타입 추가가 용이한 유연한 구조

"Simple, Decoupled, Stable"

가독성 높은 코드는 비즈니스 로직의 명확성에서 시작됩니다.

GAMEPLAY LOOP

미션 수락은 곧바로 별도의 퀘터뷰 디펜스 미니게임으로 이어집니다.



생존 성공 (Succeeded)

생존 타이머 만료 → `OnSurvived` 브로드캐스트 → 미션이 `SuccessMission()` 호출

HP 소진 (Failed)

몬스터 피격으로 HP 0 도달 → `HandlePlayerDefeated()` → 미션이 `FailedMission()` 호출

ARCHITECTURE & COMMUNICATION FLOW

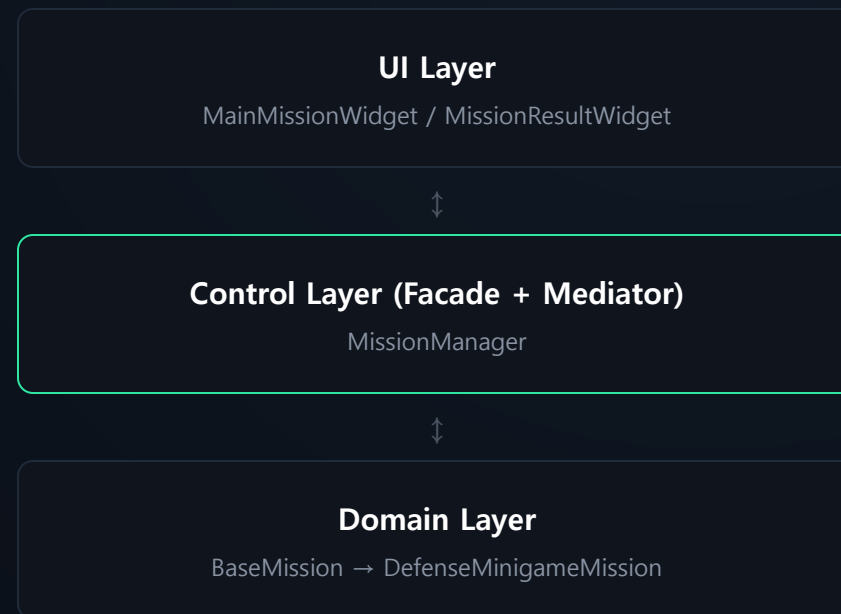
3-Layer + Facade/Mediator Structure

Widget과 Mission은 서로를 전혀 모르는 상태로 유지되며, **MissionManager**가 모든 통신을 중재합니다.

명령 흐름 (Command): 상위(Widget) → 중재자(MissionManager) → 하위(BaseMission)
로의 함수 호출

이벤트 흐름 (Event): 하위(BaseMission) 상태 변경 → C# event 브로드캐스트 → 상위(Widget) 갱신

디펜스 미니게임(몬스터 스폰 · 생존 타이머)은 MissionManager가 아니라, 미니게임을 사용하는 미션(`DefenseMinigameMission`) 자신이 소유합니다 — 단일 책임 원칙(SRP)에 따라 "어떤 미션이 미니게임을 쓰는가"는 미션 스스로 결정합니다.



CORE DESIGN PATTERNS



Facade / Mediator Pattern

MissionManager가 모든 시스템 간 통신의 유일한 진입점 역할을 하며, 객체 간 직접 참조를 제거합니다.



Observer Pattern

C# event로 상태 변경을 전파하는 Event-driven 방식으로, 매 프레임 폴링 비용을 줄입니다.



Factory Method Pattern

MissionFactory가 MissionUnique를 각 미션 클래스로 매핑, 새 미션 추가 시 분기만 추가하면 됩니다.



State Machine

Ready→InProgress→Succeeded/Failed 전이를 SetState() 한 곳에서 관리해 흐름을 예측 가능하게 만듭니다.



Object Pool Pattern

몬스터를 Destroy 대신 Deactivate로 반환·재사용해, 반복 스폰의 생성 비용과 GC 부담을 줄입니다.



Template Method Pattern

DefenseMinigameMission이 공통 흐름을 정의하고, 각 미션 서브클래스는 Enter*State() 혹은 오버라이드합니다.

MISSION STATE & TERMINATION RULES

미션 상태 (STATE)	트리거	결과
Ready (대기)	Agree 버튼 클릭	<code>AgreeMission()</code> → InProgress 진입, 디펜스 미니게임 시작
Ready (대기)	Disagree 버튼 클릭	<code>DisagreeMission()</code> → 미션 취소
InProgress (진행 중)	생존 타이머 만료	<code>SuccessMission()</code> → Succeeded
InProgress (진행 중)	HP 0 도달	<code>FailedMission()</code> → Failed

진행 중(InProgress)에는 버튼으로 미션을 강제 종료할 수 없습니다 — 오직 디펜스 미니게임의 결과만으로 미션이 종료됩니다. 과거 존재했던 수동 Success/Fail 버튼은 이 원칙에 따라 제거되었습니다.

AUTOMATED TEST CYCLE

사람이 직접 플레이하지 않아도, 미션 전체 흐름을 자동으로 검증하는 도구입니다.



F8 키 한 번으로 전체 사이클 실행

NPC 접근 → 수락 → 디펜스 미니게임 → 결과 확인까지 자동 진행



기존 코드는 자동 모드의 존재를 전혀 모름

`MissionManager`가 노출하는 public 함수만 호출 — 실제 게임 로직과 완전히 분리된 구조



Coroutine 기반 프레임 단위 이동

yield return null로 매 프레임 갱신되어, 타이머 기반 이동에서 흔한 프레임 드랍 문제 자체가 생기지 않는 구조



반복 수동 테스트 제거

전체 플로우 회귀 확인에 드는 시간 단축

```
> F8 → ToggleAutoCycleMode()
| Ready → HandleMissionButtonAction(Left)
| InProgress (자동 회피 이동)
| Succeeded → HandleMissionResultConfirmed()
| 한 사이클 종료 → 자동 OFF
```

"직접 만든 시스템은, 직접 검증할 수 있어야 합니다."

MissionAutoCycleController - Manager/

THANK YOU

안정적이고 확장 가능한 코드, 팀의 자산이 되는 개발자가 되겠습니다.

✉ `hello1013@naver.com`

🔗 `https://github.com/inhopark/MissionSystem\_Unity.git`